Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science
Semester	V
Subject Code & Name	CS3807 – Deep Learning Laboratory
AY	2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

3. Explain the convolution operation.
4. Compute output dimensions using stride and padding.
5. Visualize feature maps.
6. Implement max pooling and average pooling.
7. Calculate CNN parameters.
8. Build a CNN for image classification.
9. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer

- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X: Input Image
- K: Kernel (Filter)
- Y: Feature Map

The output feature map size is

$$\left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

where

- N: Input Size
- F: Filter Size
- P: Padding
- S: Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied elementwise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore, Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.

- Plot class distribution.

]

The CIFAR-10 dataset was successfully loaded with 50,000 training samples and 10,000 testing samples. The shape of a single image is $(3, 32, 32)$ and the training data shape is $(50000, 3, 32, 32)$. The dataset contains 10 classes.

Figure 1: Ten sample images from CIFAR-10

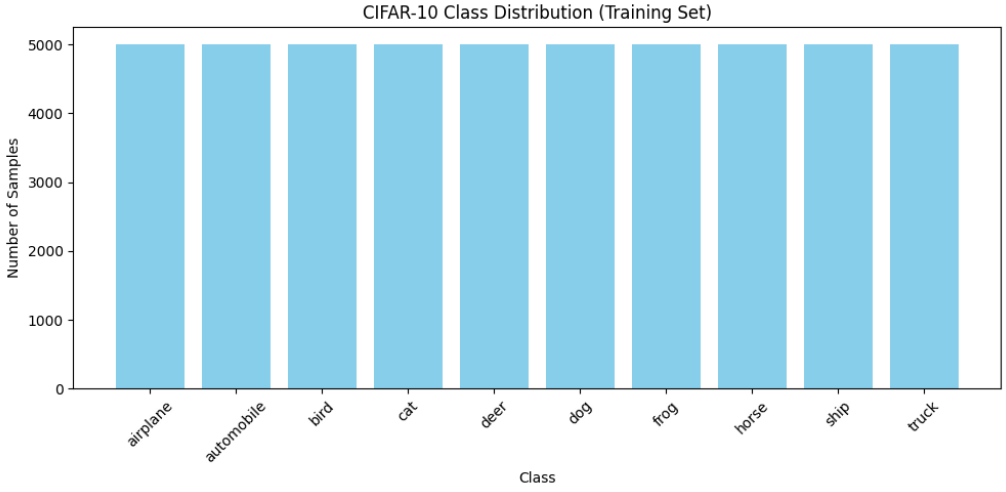

Figure 2: Class distribution of CIFAR-10 training data

Task 2

Implement a convolution layer.

Compare

- Kernel $= 3 \times 3$
- Kernel $= 5 \times 5$

- Kernel $= 7 \times 7$

Record feature map sizes.

]

A convolution layer was implemented and tested using kernel sizes of 3×3, 5×5, and 7×7.

- 3×3 Kernel: $(8, 30, 30)$
- 5×5 Kernel: $(8, 28, 28)$
- 7×7 Kernel: $(8, 26, 26)$

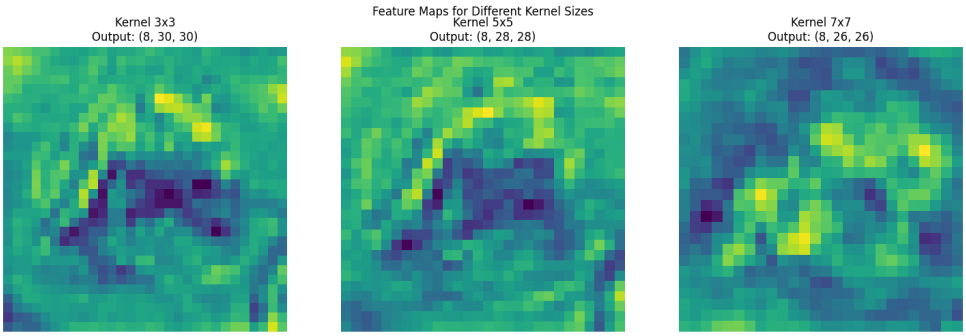

Figure 3: Feature maps for different kernel sizes

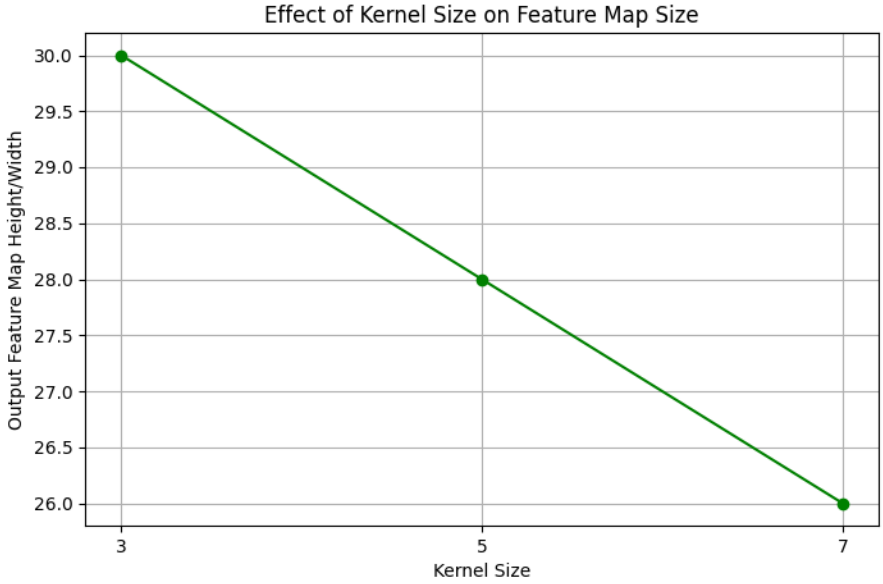

Figure 4: Comparison of kernel size and feature map size

Inference: As the kernel size increases, the spatial dimensions of the feature map decrease. The number of output channels remains fixed at 8.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

]

The effect of stride and padding on the output feature map size was studied using a 3×3 kernel.

- Stride = 1: $(8, 32, 32)$
- Stride = 2: $(8, 16, 16)$
- Padding = Same: $(8, 32, 32)$
- Padding = Valid: $(8, 30, 30)$

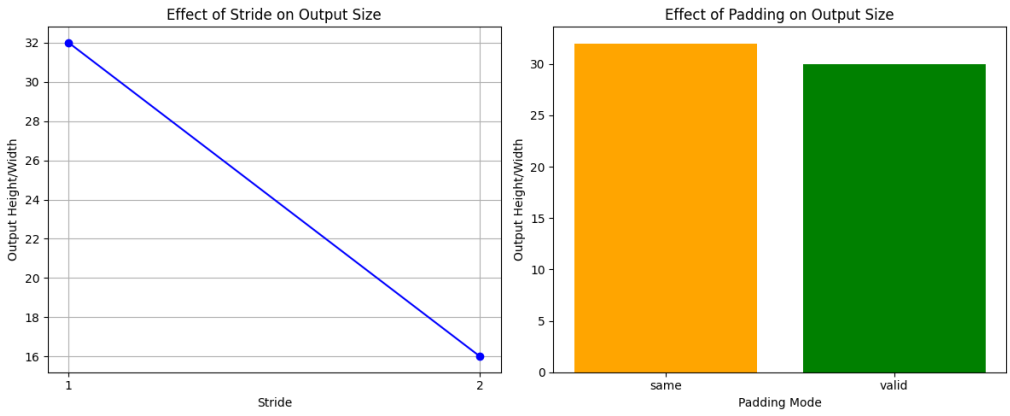

Figure 5: Comparison of stride and padding

Inference: Increasing the stride from 1 to 2 reduces the spatial dimensions of the feature map. Same padding preserves the input dimensions, while valid padding reduces them.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

]

The output of the first convolution layer consists of 16 feature maps of size 32×32. Eight feature maps were visualized to observe the features extracted by the convolution layer.

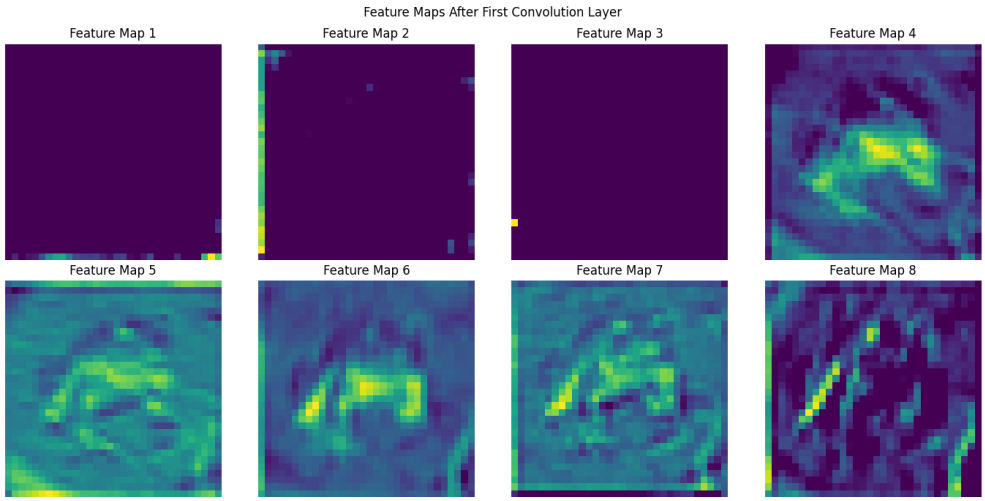

Figure 6: Feature maps after the first convolution layer

Inference: The feature maps represent different features extracted from the input image by the convolution filters. Each map highlights different patterns detected by the corresponding filter.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

]

Max Pooling and Average Pooling were compared using a 2×2 pooling window with stride 2.

- Max Pooling Output: $(1, 16, 16, 16)$
- Average Pooling Output: $(1, 16, 16, 16)$
- Max Pooling Test Accuracy: 58.68%

- Average Pooling Test Accuracy: 54.62%

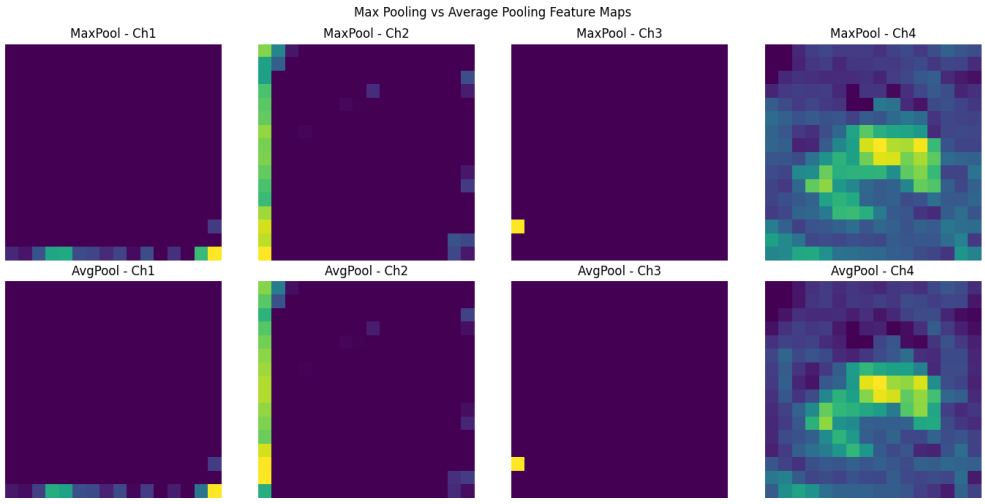

Figure 7: Comparison of Max Pooling and Average Pooling feature maps

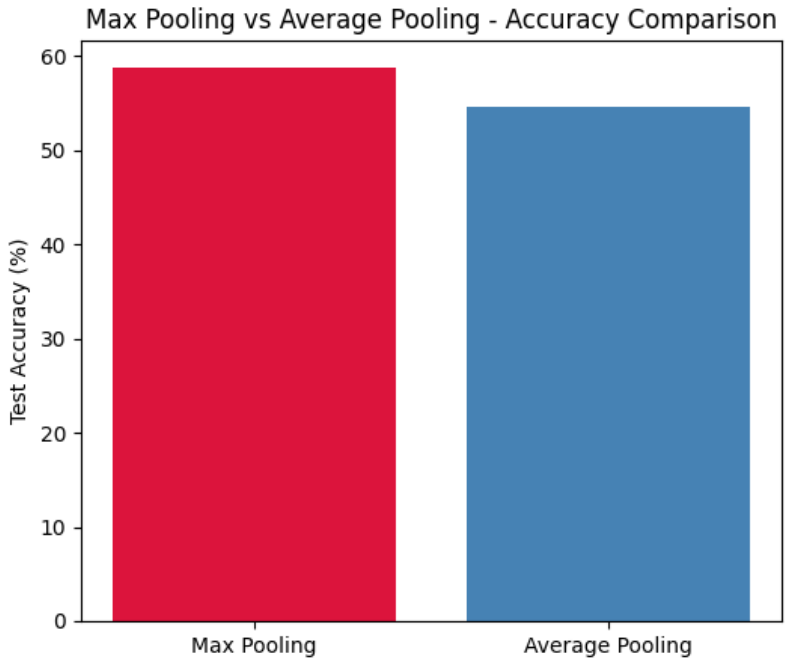

Figure 8: Accuracy comparison of Max Pooling and Average Pooling

Inference: Both pooling methods produce the same output size. Max Pooling achieved higher test accuracy (58.68%) compared to Average Pooling (54.62%).

Task 6

Construct the following CNN.

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

]
The CNN was constructed with the following architecture:

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

- Conv1: 32 filters, 3×3 kernel
- Conv2: 64 filters, 3×3 kernel
- Optimizer: Adam
- Epochs: 20
- Batch Size: 32

The model was trained for 20 epochs using the Adam optimizer. The training accuracy increased from 48.70% in the first epoch to 82.39% in the final epoch, while the final test accuracy was 69.83%.

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

]
The trained CNN was evaluated on the CIFAR-10 test dataset.

- Accuracy: 69.83%
- Precision: 70.28%
- Recall: 69.83%
- F1-score: 69.84%

9. Mandatory Plots

9.1 Sample Images

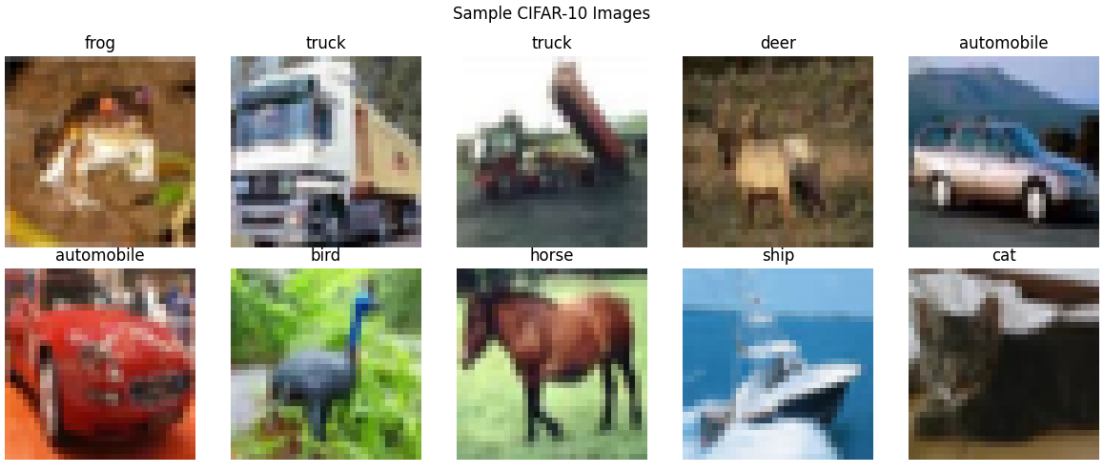

Figure 9: Sample CIFAR-10 Images

Inference: The sample images show representative examples from the CIFAR-10 dataset belonging to different object classes. The images demonstrate the visual diversity of the dataset and provide an initial understanding of the classification task.

9.2 Class Distribution

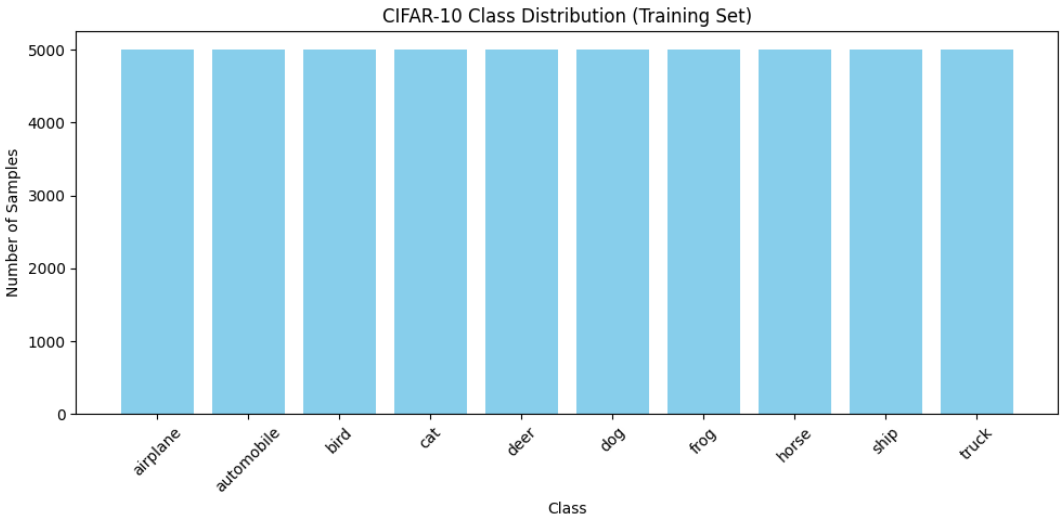

Figure 10: CIFAR-10 Class Distribution

Inference: The class distribution shows the number of training samples available for each CIFAR-10 class. The classes are approximately balanced, ensuring that no particular class dominates the training process.

9.3 Training Accuracy

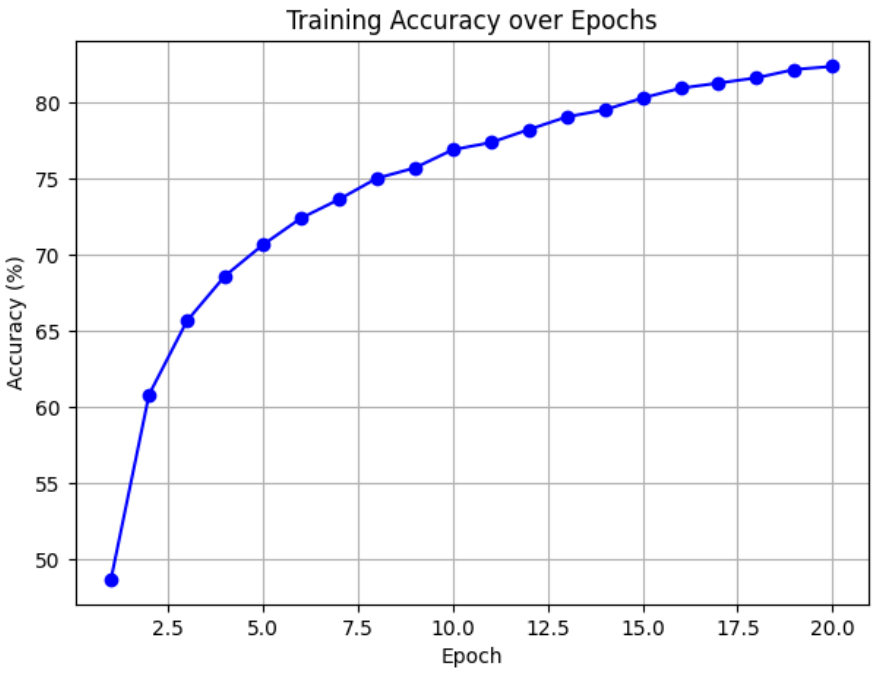

Figure 11: Training Accuracy over Epochs

Inference: The training accuracy increases consistently from 48.70% in the first epoch to 82.39% in the twentieth epoch. This indicates that the CNN successfully learns useful features from the training data.

9.4 Validation Accuracy

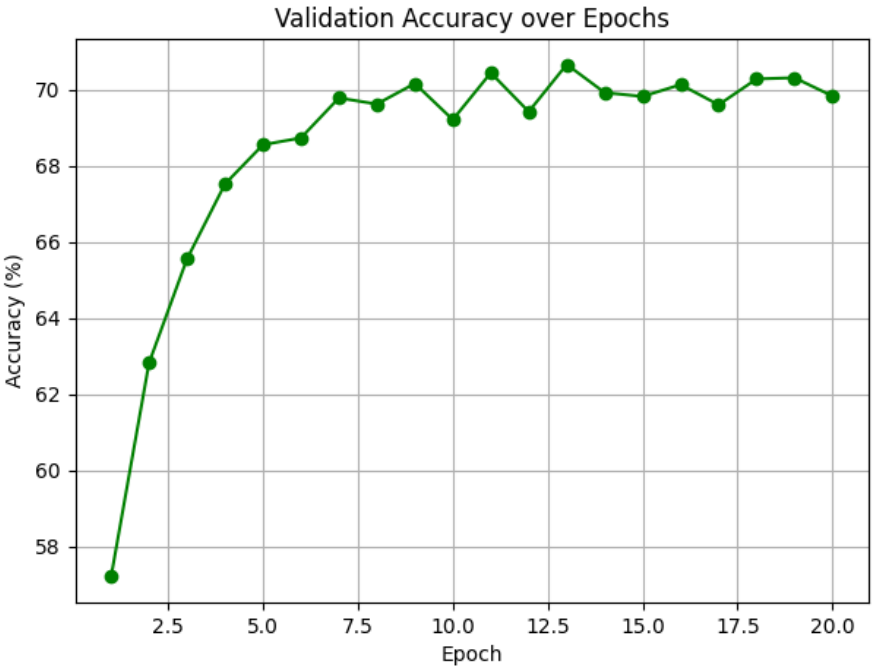

Figure 12: Validation Accuracy over Epochs

Inference: The validation accuracy improves from 57.20% in the first epoch to a maximum of about 70.66% during training, with a final accuracy of 69.83%. The small fluctuations in later epochs indicate that the model's generalization performance has largely stabilized.

9.5 Training Loss

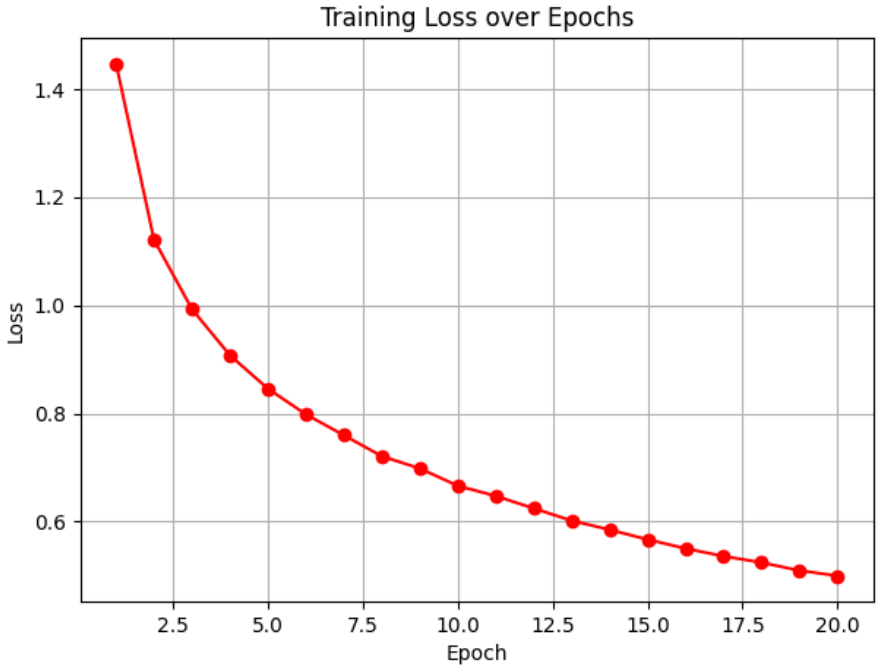

Figure 13: Training Loss over Epochs

Inference: The training loss decreases steadily from 1.4481 in the first epoch to 0.4992 in the twentieth epoch. This shows that the model is progressively reducing its classification error and learning better representations from the training data.

9.6 Feature Maps

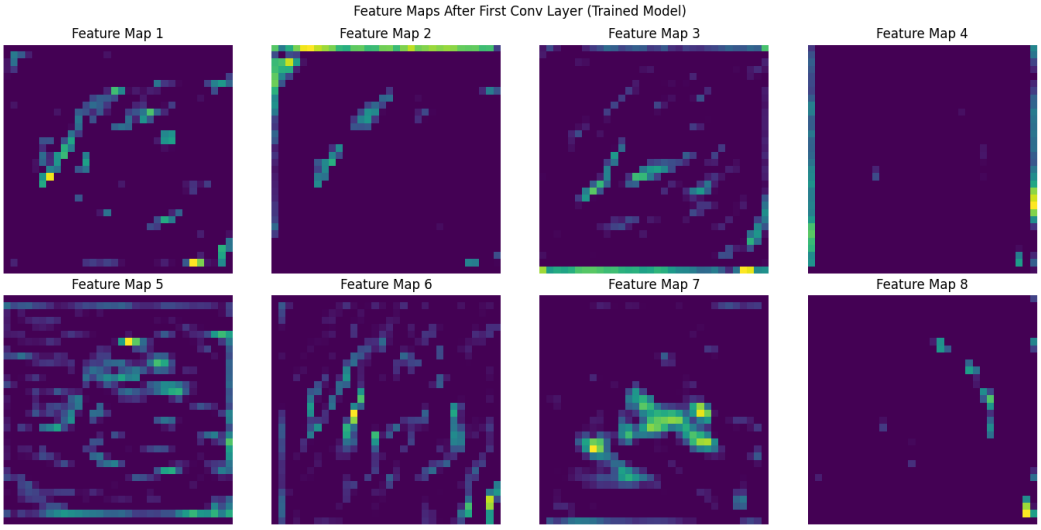

Figure 14: Feature Maps After First Convolution Layer

Inference: The feature maps visualize the activations produced by the first convolutional layer of the trained CNN. Different feature maps highlight different visual patterns and structures learned from the input image.

9.7 Confusion Matrix

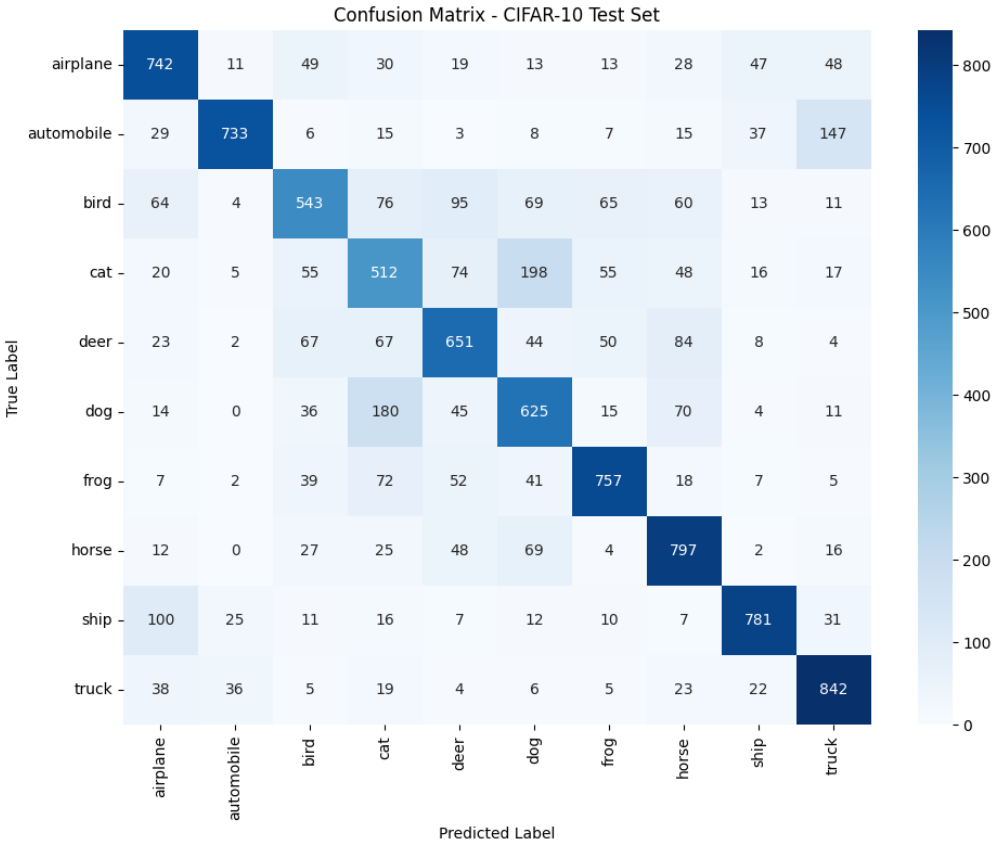

Figure 15: Confusion Matrix for CIFAR-10 Test Set

Inference: The confusion matrix shows the number of correct and incorrect predictions for each CIFAR-10 class. It helps identify classes that are classified correctly and classes that are frequently confused with one another.

10. Additional Exercises

1. **Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.**

$$\text{Output Size} = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = 32$$

Therefore, the output size is $\boxed{32 \times 32}$.

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.**

   Since RGB input has 3 channels,

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 64$$

$$= 28 \times 64 = \boxed{1792}$$

   Therefore, the convolution layer has **1,792 trainable parameters**.

3. **Compare ReLU and Sigmoid activation functions.**

Property	**ReLU**	**Sigmoid**
Range	$[0, \infty)$	$(0, 1)$
Computation	Simple	More complex
Training	Faster	Slower
Vanishing Gradient	Less likely	More likely

   ReLU is generally preferred in hidden layers of CNNs because it trains faster and reduces the vanishing-gradient problem compared with Sigmoid.

4. **Replace Max Pooling with Average Pooling and compare the results.**

   Both Max Pooling and Average Pooling produced an output size of 16×16. Max Pooling achieved a test accuracy of 58.68%, while Average Pooling achieved 54.62%.

   Thus, Max Pooling performed better for the given experiment.

5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**

   Increasing the number of filters from 16 to 64 allows the CNN to learn more features and may improve classification accuracy. However, it also increases the number of parameters and computational cost, resulting in longer training time. The exact accuracy and computation-time comparison was not measured in this experiment.

11. Results

Table 1: CNN Model Performance

Metric	**Value**
Training Accuracy	82.39%
Testing Accuracy	69.83%
Precision	70.28%
Recall	69.83%
F1-score	69.84%
Number of Parameters	60,362

12. Discussion

1. **Why is convolution preferred over fully connected layers for images?**

   Convolution is preferred because it preserves the spatial structure of images and extracts local features such as edges, textures, and patterns using shared filters.

2. **How does stride affect the feature map size?**

   Increasing the stride moves the filter by larger steps, reducing the spatial dimensions of the output feature map. For example, increasing the stride from 1 to 2 reduces the output size.

3. **What is the role of padding?**

   Padding adds extra pixels around the input image. It helps control the output feature-map size and allows the spatial dimensions to be preserved when using "same" padding.

4. **Why is pooling used?**

   Pooling reduces the spatial dimensions of feature maps, which decreases computation and helps retain the most important features.

5. **How do feature maps represent image characteristics?**

   Feature maps represent the features detected by convolutional filters. Different filters can detect different patterns such as edges, textures, corners, and other visual characteristics.

6. **Why do CNNs require fewer parameters than MLPs?**

   CNNs use local connectivity and shared convolutional filters instead of connecting every neuron to every pixel. This significantly reduces the number of trainable parameters compared with fully connected MLPs.

13. References

1. Goodfellow et al., *Deep Learning.*
2. Bishop, *Pattern Recognition and Machine Learning.*
3. Haykin, *Neural Networks and Learning Machines.*
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.